

Introduction to Linux (1)

Linux and basic terminal commands:

`ls`, `cd`, `mkdir`, `rm`, ...

Nikolaj Lindberg
nikolaj.lindberg@ling.su.se

Part 1 of 2

In the follow two lectures and labs, you'll learn how to:

- ▶ Use basic Linux commands
 - ▶ copy and move files, etc
 - ▶ move between directories (folders)
 - ▶ Edit files in a terminal editor
 - ▶ Log in to a remote Linux server
-

All done the old school way: using nothing but the terminal — only using the keyboard

Overview: Course material

Course material: Canvas → Modules → Introduction to Linux

- ▶ Slides:
 linux_1.pdf (this file!),
 linux_2.pdf (next lecture)
- ▶ Compendium of commands:
 linux_commands.pdf
- ▶ Exercises with answers:
 linux_1_exercises.pdf, linux_1_solutions.pdf,
 linux_2_exercises.pdf, linux_2_solutions.pdf

Tip: download PDF files and view in local reader (rather than the built in Canvas reader)

Overview: Today's topics

- ▶ Short description of Linux
- ▶ The Linux terminal ('shell')
- ▶ Commands, arguments, flags
- ▶ The file tree (directory structure)

Linux: Welcome the 1970's!

The *Linux* operating system (OS) is descending from *Unix*, originally developed in the 1970's. Being old is a *positive* in this case

The term *Linux* can refer to different things:

- ▶ The core functions ('kernel') of the OS
 - ▶ The kernel combined with user programs ('GNU/Linux')
 - ▶ Different *distributions* of systems (Debian, Ubuntu, ...)
-

Ubuntu Linux is popular for server, desktop and laptop computers. It includes everything you need for programming.

Linux is everywhere!

The internet runs on Linux: There are *millions* of Linux internet servers

Linux is popular in computer science and many other areas of research

Linux is popular in cloud computing and 'big tech'

Android mobile phones and tablets run Linux

In other words: Linux is *relevant* and *useful*.

Side-note: The Mac is also 'Unix'

Darwin (in Apple's macOS) is also a Unix-like operating system.

The terminal of a Mac computer is similar to the Linux terminal you will use in this course.

It's you, a terminal and a keyboard

To a beginner, the Linux terminal may seem

- ▶ old school (It is)
 - ▶ confusing (well...)
 - ▶ primitive, impractical (No!)
-

Unix/Linux was not created to be beginner friendly

The terminal is powerful: Simple Linux commands can be combined to perform complex tasks (not the focus of this course, but we'll touch upon it)

The trick is to become as one with the machine: you need to put in some time and effort. With time, you'll see the beauty of this approach.

Don't fear the command line

A few calming words:

If you're not familiar with using a command line (terminal), it may be frustrating at first.

Try to avoid thinking that you are stupid. You are not expected to know these things prior to this course.

It's fine to be confused; it's part of learning.

This is a skill, and it takes some practice.
Soon, things will fall into place. (I hope...)

What's the terminal?

The terminal is a window, in which you run the *shell* — the software that speaks to the computer.

You type a command at the *prompt* and hit the return key (). The prompt is the line at the bottom of the terminal ending with a \$.

Example:

Remove a file called `wf.py`:

```
$ rm wf.py 
```

The dollar sign — the prompt — is not part of the command, and will be left out in most examples below.

Most important today: The basic Linux commands

<code>cd</code>	change directory
<code>ls</code>	list files
<code>mkdir</code>	make new directory
<code>cp</code>	copy files
<code>mv</code>	move files
<code>rm</code>	remove files
<code>pwd</code>	print working directory (where you currently are)
<code>ssh</code>	remote login ('secure shell')
<code>...</code>	and a few more, see linux_commands.pdf

Do you see a pattern? (Economy of typing)

A command and its *arguments* (*terminology*)

A command might take *arguments*. Here is how to move to your `public_html/` directory using `cd`:

```
cd public_html/
```

← `cd` is the command, `public_html/` is its argument

The `mv` command takes *two* arguments, in this case file names. This is how you rename a file:

```
mv ksl.txt poem.txt
```

← `ksl.txt` and `poem.txt` are arguments

The arguments are the stuff (parameters) the command needs to do its job. Different commands take different arguments.

Switches: sub-commands to a command (*terminology*)

Most commands take *switches* (*flags* or *options*) as arguments.

These can be viewed as 'sub-commands', altering the behaviour of a command:

<code>ls</code>	<i>(list files)</i>
-----------------	---------------------

<code>ls -s</code>	<i>(... along with file size)</i>
--------------------	-----------------------------------

Some commands let you combine switches:

<code>ls -s -h</code>	<i>(file size in "human readable" format)</i>
-----------------------	---

<code>ls -sh</code>	<i>(same as above)</i>
---------------------	------------------------

Basic command line syntax (*spaces between tokens*)

The command always comes first, before its arguments.

There is always space between a command and arguments:

```
rm file1.txt file2.txt
```

Switches start with a hyphen, sometimes two. They are delimited by space:

```
ls -a
```

```
man --help
```

No space between hyphen and argument.

Let's try some simple commands

Log in to the computer in front of you.

Click the magnifying glass search icon (🔍) at the top right of the screen.

Type

```
terminal
```

Start the terminal by clicking it.

Try a couple of commands, followed by return (enter), :

```
ls 
```

```
ls Desktop 
```

(From now we'll leave out  in most examples.)

Some simple commands (*no arguments*)

<code>pwd</code>	What's my current directory?
<code>hostname</code>	What computer is this?
<code>uname</code>	What operating system?
<code>date</code>	Date and time?
<code>ls</code>	What directories and files are in the current directory?

`ctrl` + `C` Cancel — to get out of a mess

`ctrl` + `D` Oops! Now you'll have to start again 🙄 (exits terminal)

You will use a common remote server for much of the programming course:

`triton.dsv.su.se`

You can reach this server from your own computer, given that you can run the `ssh` command.

Using ssh to log in to triton.dsv.su.se

- ▶ ssh <SU-ID>@triton.dsv.su.se
- ▶ Enter your password, then hit .
- ▶ Answer yes to the question (once for each machine you log in from).

<SU-ID> should be replaced by your user name.

Example: User `kali2818` logs in:

```
ssh kali2818@triton.dsv.su.se
kali2818@triton.dsv.su.se's password:
```

While entering your password, you'll get no response or feedback.
Just keep typing and then hit return.

Terminal interaction on the triton server (1/2)

The terminal prompt of user kali2818 on the triton server:

<USER NAME> @ <SERVER NAME> : <CURRENT DIRECTORY> \$

kali2818@triton:~\$ ← user name @ server name : current dir \$

User kali2818 runs the uptime command:¹

kali2818@triton:~\$ uptime ↵

12:37:48 up 79 days, 6:02, 5 users, load average: 2,00, 2,00, 2,00

Try it yourself!

¹uptime tells you how long the system has been running.

Terminal interaction on the triton server (2/2)

User kali2818 creates a new directory, intro_to_prog:

```
kali2818@triton:~$ mkdir intro_to_prog
```

User kali2818 moves to the new directory, and the prompt is updated:

```
kali2818@triton:~$ cd intro_to_prog/  
kali2818@triton:~/intro_to_prog$
```

Advanced: It is possible to re-configure your prompt if you don't like the current look of it. (You'll have to figure it out for yourself.)

Some simple commands (*no arguments*) — same as above

<code>pwd</code>	Where am I?
<code>hostname</code>	What computer?
<code>uname</code>	What OS?
<code>date</code>	Date and time.
<code>ls</code>	List directories and files

`ctrl` + `C` Cancel — to get out of a mess

`ctrl` + `D` Oops! Now you'll have to start again 😞 (exits terminal)

Understanding the hierarchical file system, the *file tree*

- ▶ Files are kept in *directories* (“folders”)
- ▶ Directories may include subdirectories
- ▶ Directories are organised in a *tree structure*

You move between directories using the `cd` command.

When you log in, you end up in your personal *home directory*.

Your home directory sits a few steps down in the file tree.

Your home inside the computer: ~ (*the tilde character*)

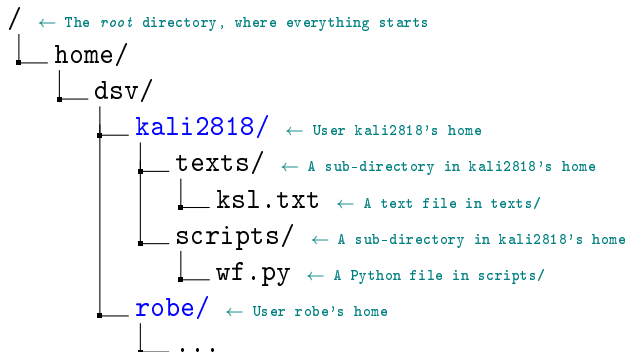
- ▶ Each user has their own *home directory*. This is where you keep your files.
 - ▶ The name of your home directory is identical to your user name (SUKAT-ID).
 - ▶ When logging in, you end up in your home directory
-

The home directory is located a few steps down in the file tree.
This is the path to user `kali2818`'s home on `triton.dsv.su.se`:

```
/home/dsv/kali2818/
```

There is a shorter way to write the path to your home: ~

Example file tree paths (abstract rendering)

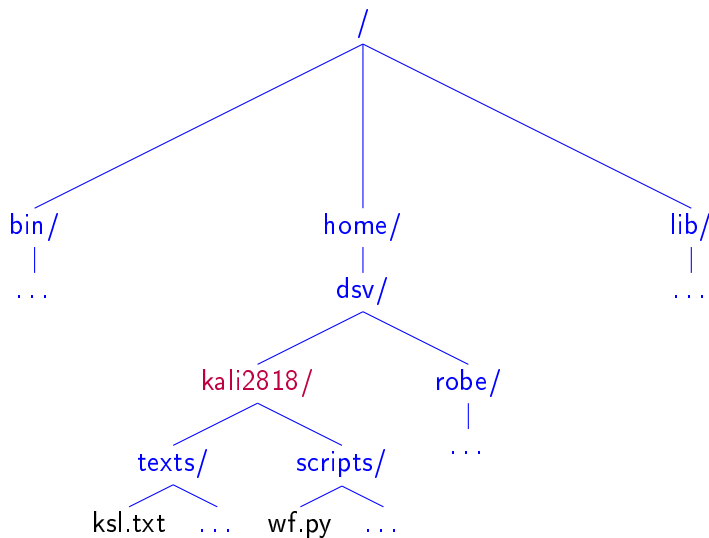


`kali2818` and `robe` are different *users*, with different *home directories*

The full path to the file `wf.py`:

`/home/dsv/kali2818/scripts/wf.py` ← This is what a path actually looks like

Sketch of part of a possible file tree (abstraction)



Full vs relative path (1/2)

A full path starts at the root directory, /:

```
/home/dsv/kali2818/public_html/index.html
```

A relative path never starts with /:

```
public_html/index.html
```

Full vs relative path (2/2)

Each file has a unique *file path*, the complete name of the file. The path below points to the file `ksl.txt` found in the `texts` directory in `kali2818`'s home directory:

```
/home/dsv/kali2818/texts/ksl.txt
```

If user `kali2818` is currently in their home directory

```
/home/dsv/kali2818/
```

the (relative) path to `ksl.txt` is

```
texts/ksl.txt
```

Full vs relative path: Example

Assuming that you are in your home directory:

List the contents of your own `public_html/` directory:

```
ls public_html/    ← Probably empty!
```

List the contents of `kali2818`'s `public_html/` directory:

```
ls /home/dsv/kali2818/public_html/  
b_j_e.jpg          deeper_understanding.txt ksl.txt  
computer_world.txt index.html
```

Full path is independent of where you are. Relative path is from where you are.

cd: Change directory

You move between different directories — typically sub-directories in your home dir — using the `cd` command:

```
cd public_html/ ← Move to sub-directory public_html/
```

```
cd .. ← Move back out from public_html/ Two dots!
```

As you move around, relative paths to directories and files change.

`..` means the directory above.

`../..` means two directories above, etc.

cd without any argument takes you home

When you run the `cd` command without supplying a path to a directory, you'll return to your home directory.

Return to your home directory:

```
cd
```

ls: Listing files

User kali2818 is in their home directory.

ls without any arguments lists directories and files in the current directory:

```
ls                                     ← No arguments
public_html scripts texts           ← User kali2818 has three directories
```

List contents of a sub-directory:

```
ls texts/
computer_world.txt      fk.txt      sl.txt      trsc.txt
deeper_understanding.txt ksl.txt     starfish_and_coffee.txt
```

ls: Same example with full path

If you are not in [kali2818](#)'s home directory, you can use a full path:

```
ls /home/dsv/kali2818/texts/  
computer_world.txt      fk.txt      sl.txt      trsc.txt  
deeper_understanding.txt ksl.txt    starfish_and_coffee.txt
```

Shorter full path version using ~:

```
ls ~kali2818/texts/
```

...: The directory above

List the contents of the directory *above* the current directory:

```
ls ..
```

Two directories above:

```
ls ../..
```

etc.

Experiment with relative paths yourself!

ls: Wild card character *

Match several directory or file names: *

List all files in `texts/` that end in `.txt`:

```
ls texts/*.txt
```

You'll have to explore file name matching on your own!

File extensions: signaling file type

The file name `ks1.txt` has the `.txt` *file extension*.

A file extension is arbitrary, but is conventionally used to signal file contents: `.txt` indicates a plain text file.

You don't *have* to use the `.txt` extension—or *any* extension—when you create a text file. (If you want to be reasonable, it is a good idea.)

Examples of file extensions:

`.txt .csv .wav .sh .pdf .py`

Program files written in the Python programming language typically use the `.py` extension.

Repetition: the file tree

To repeat:

- ▶ There is one root directory, /
 - ▶ A directory may have sub-directories
 - ▶ Each file resides in a directory
-

There is a unique *path* to each file.

- ▶ A *full* path starts at the *root* (/), e.g.:

```
/home/dsv/kali2818/scripts/wf.py
```

- ▶ A *relative* path starts from a given place in the file tree, e.g. from inside `/home/dsv/kali2818/`:

```
scripts/wf.py (no need to type full paths all the time)
```

- ▶ On the Canvas course page (canvas.su.se), you'll find an Introduction to Linux module
- ▶ There is a list of commands to learn in the file `linux_commands.pdf`
- ▶ There are exercises in the file `linux_1_exercises.pdf`
- ▶ There are answers to the exercises in `linux_1_solutions.pdf`
- ▶ These slides, `linux_1.pdf`, are found under Slides.

Let's take a look at these things

Now on to the exercises

- ▶ List of commands needed for the exercises:
`linux_commands.pdf` (`canvas.su.se`)
- ▶ Exercises `linux_1_exercises.pdf` (`canvas.su.se`)
- ▶ Answers `linux_1_solutions.pdf`